SHoEbox CPU optimizations

13 November 2026 | Full reference-v1 | Base a798c92

The default full-reference CPU path is optimized. Five paired continuations measured 1.35x repeated-text training throughput at 1M parameters and 6.32x warm-inference throughput. All 231 distinct tests passed; paired logits, NLL and fitted parameters match bit for bit.

Matched throughput

Workload	Targets / pairs	Reference B/s	Optimized B/s	Speedup
1M repeated training	1,024 / 5	128.2	173.1	1.35x
1M natural-text training	1,024 / 5	200.5	229.1	1.14x
1M warm inference	4,096 / 5	419.2	2,650.5	6.32x
5M repeated training	1,024 / 5	96.2	119.1	1.24x
1M longer training	4,096 / 1	53.5	75.3	1.41x
5M longer training	2,048 / 1	63.9	82.8	1.29x

Rates are bytes/s. The second column is targets per continuation / paired repetitions. Each process restores the same actual a798c92 checkpoint after 1,024 training targets; process order alternates. Models contain 1,031,952 or 4,998,928 parameters, with 12 or 64 experts per region, width 64, seed 7 and all 18 regions. Periodic training uses LR 0.04 and local LR 0.04/3. Timing includes logit emission and excludes compilation, initialization, checkpoint I/O and hashes. CPU 0 is pinned; Intel Xeon Platinum 8573C. GCC 14.2.0 uses -O3 -march=native -std=c++17. Natural text is the same FineWeb byte stream used in the profile; input hashes are retained.

Run-to-run timing variation remains visible in the retained raw results. The first four rows report medians of five pairs; the longer continuations are single pairs. These are the full-reference workloads, with every original teaching objective and ancestor visit retained.

CPU worker comparison

The initial three-pair worker comparison favored serial execution. Extra workers consumed more CPU time and reduced throughput on this workload, so the default remains one worker. The persistent team is available through model.configure_content_workers(count, cpus) for larger ready batches; it does not change the learning policy.

Workers	Median B/s	CPU seconds / 1,024 targets
1	170.1	6.01
2	155.3	13.17
4	153.4	26.66

Implementation and verification

Area	Change
Semantic allocation	Reuse an O(1) free-slot inventory; collect only when exhausted, with all existing live owners and growth retained.
Pending credit	Index ticks directly; maintain region/tick/depth pending sets. Preserve descending input credit, ascending neural credit, and delayed-cycle behavior.
Reusable storage	Keep lazy per-bank gradients and bounded request queues; recycle local-credit map nodes through a model-owned allocator.
Immutable caches	Cache generation-checked input norms, role transforms and up to 64 root schedules of at most 4,096 handles each. Rebuild on import/reset.
Content kernels	Batch ready VJPs by bank; tile up to 64 weight-gradient requests per bank. Preserve each request's clipping, gate, rate and per-element reduction order.
CPU workers	Provide a persistent team for disjoint gradient rows in batches of at least 32 requests. Keep parameter mutation and bank flushes ordered.

Correctness and checkpoint continuation

The original 224 package regressions and seven new tests passed. All seven new tests also passed with 64 experts. The reference lifecycle and focused parallel-kernel ASan/UBSan runs passed. The reference acceptance mechanisms passed credit coverage, primary clocks, frozen downstream interventions, generation parity and exact chunked continuation. These components ran in separate invocations; the mechanism-only driver retains its explicit partial-status label, and optimization_validation.json records the combined passing evidence.

Checkpoint version 39 and composition ABI 10 are unchanged. Derived caches and workers are transient. The new allocator can change physical slot indices and full-state fingerprints relative to the old allocator. Paired outputs and all fitted state remain exact. Optimized save/load and chunked continuation are bitwise exact, including after collection and slot recycling, and live DAGs beyond the initial 16,384 slots.

Reproduce

```
python experiments/identity_semantics/src/compare_optimized_runtime.py \
  --baseline-runtime /path/to/base/experiments/identity_semantics/runtime \
  --repeats 5 --output /tmp/shoebox-comparison.json
python -m unittest discover -s experiments/identity_semantics/tests -v
SHOEBOX_EXPERTS=64 python -m unittest discover \
  -s experiments/identity_semantics/tests -p test_optimizations.py -v
python experiments/identity_semantics/tests/run_completion_sanitizer.py
python experiments/identity_semantics/tests/run_optimization_sanitizer.py
```

The comparison also accepts --mode eval, --corpus and --worker-counts. Raw timing, test-ID coverage and sanitizer evidence are in experiments/identity_semantics/results/optimization_*. The code package excludes trained models, datasets and compiled binaries. GPU execution remains a separate backend project.